esiea
INGÉNIEUR·E·S D'UN NUMÉRIQUE UTILE

IoT Conso
Labs Sessions

IoT and energy consumption
The ESP32 use case

Information and Assessment . . . 2

1 Discovery of ESP32 **5**
1.1 IDE: PlatformIO or Arduino . . . 5
1.2 ESP32 Platform . . . 6
1.3 Communication . . . 6
1.4 Digital Input/Output . . . 7
1.5 ADC – Analog-to-Digital Converter . . . 8

2 Acquire external data, push data to the world **10**
2.1 Using I2C or SPI to acquire or push data . . . 10
2.2 Using API to acquire external data . . . 11
2.3 Push data to the world . . . 11
2.4 To go further (not evaluated) . . . 11

3 ESP32 and energy consumption **12**
3.1 Measuring energy consumption with Analog Discovery 2 . . . 12
3.2 Using Sleep and DeepSleep modes . . . 14

4 Need for specifications to optimize consumption **16**
4.1 What will be used? When? What response time is acceptable? . . . 16

5 Annexes **17**
5.1 Details on ADC and ESP32 . . . 17
5.2 ESP32 with BME680 . . . 17
5.3 ESP32 with API call and JSON read . . . 18
5.4 ESP32 and AdafruitIO . . . 18
5.5 ESP32 and ULP . . . 18

6 Bibliography **19**

IAg Assistance limitée

P. Courbin

Information and Assessment

This lab is not a piece of homework to hand in: it is your opportunity to understand. What you understand during the sessions is what will earn you marks. What you generate will earn you none.

What you have to submit

Ask for us to check your lab when you have finished a part. You must also submit **one file per pair**:

- on `https://learning.esiea.fr`
- **before the deadline**
- in **PDF** file format with the name: **[IoTConso][TP] Name1 & Name2.pdf**
- each day late will be penalized by **-1 point**

This file contains **three sections**, in this order, for **8 pages maximum** excluding the measurement sheet:

A. Measurement sheet. The sheet you filled in and deposited at the end of Lab Session 3, attached here unchanged.

B. Evidence file. Raw value exports and screenshots from WaveForms, photographs of your assemblies (with a sheet showing your names and the date), Fritzing schematics, serial monitor captures showing your pair prefix, and the link to your Git repository. **Your code goes in the Git repository, not in the PDF.**
This section also contains a table justifying **your pin assignment**, one line per GPIO used. *"Because it is an ADC pin"* is not a justification: state the constraint that made you keep this pin and reject another, and **give the link to the page of the documentation where you found it** — datasheet, technical reference manual or board schematic. A justification without a source is not counted.

C. Analysis note, 2 pages maximum. One imposed topic per pair, given in Lab Session 3. Written in continuous prose (no bullet lists), referring explicitly to your own figures and your own measured values, and including one thing that did not work and why.

This report is **not marked on the correctness of your answers**. What is marked is the presence and traceability of your evidence, and the quality of your reasoning in the analysis note. The analysis note is the only part on which you will receive detailed written feedback, and it is the rehearsal for your PoC report: the marking criteria are the same, and they are published on Moodle from Lab Session 1.

Consider using Fritzing for your electronic schematics. To properly display a code, you can use Hilite.me.

How your understanding is assessed

Individually, in class, on Moodle with internet access blocked:

When	On what	Duration	Closes
Start of Lab Session 2	Quiz 1 — Part 1	10 min	at 15 min
Start of Lab Session 3	Quiz 2 — Part 2	10 min	at 15 min
End of Lab Session 3	Self-test on Part 3 — **not marked**, answers shown immediately	5 min	—
Start of Lab Session 4	Quiz 3 — Parts 1 to 3: multiple choice + 2 open questions	40 min	at 45 min

Each quiz **closes [duration + 5] minutes after it opens**, so a launch problem does not cost you the assessment. It does not cover being late.

You may bring one A4 sheet of handwritten notes, prepared by you, for each quiz. Typed or printed sheets are not accepted. Writing your report seriously is the best way to build that sheet: this is where the work you actually did is given back to you.

Part 3 is the core of Quiz 3. The self-test at the end of Lab Session 3 tells you what you should be able to explain when you walk out that evening; whatever is missing goes on your A4 sheet.

The values you must bring with you

Some questions in Quiz 3 ask you to reason **about your own measurements**. You will be asked to state the value first, then to interpret it. No credit is lost for a value that differs from another pair's — they are supposed to differ.
The measurement sheet from Lab Session 3 contains everything you need. In particular, make sure you can quote:

Value	**Unit**
Your measured resistor value	**Ω**
Average current in normal mode	mA
Average current in light sleep	mA
Average current in deep sleep	mA
Peak current on wake-up	mA
Integration window used for each average	s
Button-to-LED and LDR-to-LED response times	ms
Your own T_{on}, duty cycle and T_{sleep}	ms, %, s

Copy these onto your A4 sheet. Nothing else from your file will be available to you during the quiz.

Your pair parameters

N is your pair number, given with your kit in Lab Session 1.

Parameter	**Formula**	**Where it is used**
LED on-time	$T_{on} = 50 \times N$ ms	Parts 1 and 3
Blink period	**2000 ms, the same for everyone**	Parts 1 and 3
Deep-sleep wake-up period	$T_{sleep} = 1 + 0.25 \times N$ s	Part 3
Serial message prefix	`[Binome NN - NAME1 & NAME2]`	everywhere

Every line you print on the serial monitor must start with this prefix, followed by whatever your program actually has to say. This prefix must be visible on every serial monitor screenshot in your file.

Keeping the blink period fixed at 2000 ms means everyone uses the same capture window on the Analog Discovery. Your duty cycle is $2.5 \times N$ %, so **in Part 3 no two pairs measure the same average current**.

Your GPIO assignment is free. Not every pin can do everything: interrupt capability, ADC block, RTC wake-up capability and strapping constraints all restrict the choice. Finding out which pins you can use, and why, is part of the work — and you will have to justify each choice, with its source, in your evidence file.

On the use of LLMs

Using an LLM is authorised and **must be documented**, question by question, using the pictograms recommended on Moodle: state what the tool gave you, and what you took from it. Undocumented use is plagiarism.

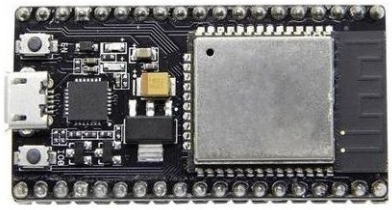
Bear in mind that the report is not where your understanding is measured. Generating it saves you nothing and deprives you of the material you will need for the quizzes. Conversely, a report written by you is exactly what you need in order to build your A4 sheet.

Some indications on the timing

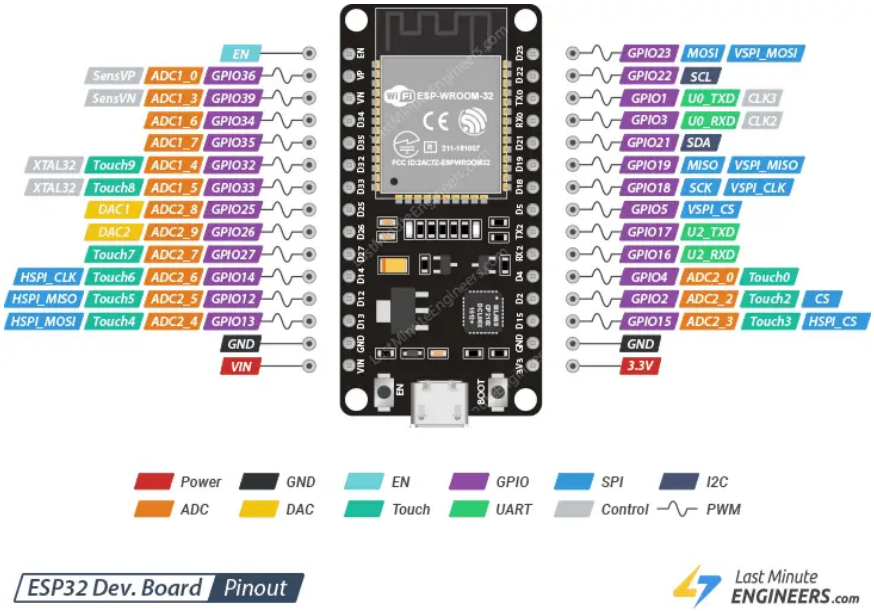
Exercises are in gray frames.

- **Lab Session 1** – You should finish Part 1. Most of it can be done at home: you have the kit, and Quiz 1 opens Lab Session 2.
- **Lab Session 2** – You should finish Part 2.
- **Lab Session 3** – You should finish Part 3. **This is the only session with the Analog Discovery 2.**
- **Lab Session 4** – You should finish Part 4 quickly to start your PoC in team.

This lab may seem long and complex. Don't hesitate to ask as many questions as you want to your teachers.



(a) NodeMCU ESP 32



(b) ESP32 Board PINOUT Reference [8]

Figure 1: Details on the board used

PART 1 | DISCOVERY OF ESP32

By the end of this part, you must be able to explain, in your own words:

- why the clock speed of a SoC matters, and what it does *not* determine;
- what happens physically on the line when two devices use different baud rates;
- what distinguishes polling from an interrupt, *at hardware level*, and what each costs;
- why the ADC range of our board is not simply 0 to 3.3 V, and what attenuation changes;
- what a voltage divider is used for **in this lab specifically**;
- which GPIO pins of your board can do what, and why: interrupt capability, ADC1 versus ADC2, RTC wake-up capability, strapping pins.

These are the concepts assessed in Quiz 1, at the start of Lab Session 2.

1.1 IDE: PlatformIO or Arduino

If you have an error concerning the driver, the Windows Drivers are available on Moodle.

1.1.1 PlatformIO

Most of the examples are given in the following GitHub repository using the PlatformIO IDE.
Please, clone the repository : `https://github.com/pcourbin-teaching/esp32_examples`
You may start by copying an example (e.g. `ESP32_Arduino_ADC`) to start the questions after Section 1.3.2.

1.1.2 Arduino IDE

We recommend using PlatformIO.
Install Arduino IDE only if you do not want to use PlatformIO described in Section 1.1.1
If you prefer to use the Arduino IDE, all examples may work **except the one using the ULP section**.

Install the IDE

- Download and install the latest version of the Arduino IDE here: `https://www.arduino.cc/en/software`
- We will use a NodeMCU ESP32 board. You need to add support for this board into your IDE. According to [15], in your Arduino IDE, you need to:
  1. open `File > Preferences` and enter `https://dl.espressif.com/dl/package_esp32_index.json` into the "Additional Board Manager URLs".
  2. open `Tools > Board > Boards Manager`
  3. Search for `ESP32` and press install button for the `"ESP32 by Espressif Systems"`.

Configure the IDE

1. Connect the board using the USB cable
2. Launch the Arduino IDE
3. Open the "blink" example: `File > Examples > 01.Basics > Blink`
4. Select your board: `Tools > Board > ESP32 Arduino > NodeMCU-32S`
   For your information, if you select the right board (`NodeMCU-32S` or even `Node32s`), then `LED_BUILTIN` is already defined as `2` since... a LED is already builtin on the digital output `D2`.

5. Select your serial port: `Tools > Port`
   As explain on the Arduino website: "This is likely to be COM3 or higher (COM1 and COM2 are usually reserved for hardware serial ports). To find out, you can disconnect your Arduino board and reopen the menu; the entry that disappears should be the Arduino board. Reconnect the board and select that serial port."
6. Upload the program: click on the `upload` button.
   The code is compiled to be readable by the microcontroller installed on the board. Then, the code is uploaded onto the board. Finally, the board restarts and the code is launched. Please, understand that *the code is then on the board and not on your computer*, so, if you connect your board to a USB Charger, the LED will continue to blink.



If you do not succeed to send your program to the board:
1. click on the `upload` button
2. when `Connecting.....` is displayed, push the `BOOT` button on your board until the process start.
3. when it is uploaded, if nothing worked, try to push the `EN` button on your board.

1.2 ESP32 Platform

1.2.1 Questions about the Platform

In this part, you will need to deepen your understanding of hardware and NodeMCU ESP32 boards by doing some research on the Internet.

1. What is the microcontroller or System on Chip (SoC) used on our NodeMCU ESP32? What is the clock speed? Why is this important?
2. Describe your board (Voltage, microcontroller, number of Digital I/O, Analog Input/Output, main components, ...) (10 characteristics maximum).
3. Explain: In which case should I use an NodeMCU ESP32 instead of an Arduino Uno?

1.3 Communication

1.3.1 Questions

When you want to communicate between the ESP32 board and your computer, you can use the "Serial" library. For information, it will send a UART "frame" to communicate through the physical connection (See [21, 12]).

1. What is a "baud rate"? Explain what happens if the computer and the board do not have the same "baud rate".
2. If I transfer data at a speed of `115200 bits / second` and I send a message of `48 bytes`, how long will it take?

1.3.2 Practice

The USB cable can be used to open a serial communication between the board and your computer. You can consider taking a look at this page: `https://docs.arduino.cc/language-reference/en/functions/communication/serial/`.

Create a program that:
1. display "`Hello Arduino World!`" every 2 seconds.
2. From now on, **every** line you print on the serial monitor must start with your pair prefix (see page 2), followed by the actual content of the message:
   `[Binome 07 - DUPOND & DUPONT] Hello Arduino World!`
   This prefix must be visible on every serial monitor screenshot in your report.

1.4 Digital Input/Output

1.4.1 Questions

1. What is a `Digital Input`? What is a `Digital Output`?
2. For `Digital Input`, explain the difference between a "polling" approach and the use of "interruptions".

1.4.2 Practice

Digital Output

You already used a "digital output" with the "blink" example. Maybe you noticed that you have 2 declared functions in the Arduino code :

setup allows configuring the ports (input/output), initialize variables, etc.
It is executed only once, when you power on the board.
`pinMode(2, OUTPUT);` configures the pin "2" to be an output. Then, we can force the pin 2 to be at the HIGH level (3.3V) or the LOW level (0V).

loop is the main function.
It is executed infinitely (`loop`...)
`digitalWrite(2, HIGH);` changes the state of the pin "2" to HIGH level (3.3V). If a LED is connected to this pin, it will light up.
`delay(1000);` does "nothing" during... 1000ms, so 1s.

Create a program that:

1. Blink a LED using **your own pair parameters** (see page 2):
   - $T_{on} = 50 \times N$ ms on
   - off for the rest of a **2000 ms** period
2. Blink a LED to display the "SOS" message using the Morse code. Beware, the Morse code precisely defines the time between each symbol, letter and word ... you must respect the protocol perfectly!

Digital Input

Now, we want to get information from the world... using a button.
Follow these steps:

1. Connect a button (with a resistor...) to one pin of your board (do not forget to give the schematic in your report).
2. In the code, configure this pin as an input.

Create a program that:

1. Without interruptions: Count the number of times the button is pressed **and** released (if you **press then release** the button, increment a counter). You will display this number on your computer using the serial connection each time it changes. (And only when it changes!)
2. Do it again but using the feature of "interruption". An interruption is the fact of asking our program to handle an event when it occurs while it is not waiting for it (our program "interrupt" itself, leaves its current line of code, will handle the event, then returns to its previous line of code). You can read this page to know how to use this principle with an Arduino board: `https://docs.arduino.cc/language-reference/en/functions/external-interrupts/attachInterrupt/`.

1.5 ADC – Analog-to-Digital Converter

Now, we want to obtain a value which is not "digital" but "analog", e.g. light, temperature, etc. Problem: computer science and embedded systems are in a digital world. Then, we have to use the **A**nalog-to-Digital **C**onverter (ADC). ESP32 boards have ADC, so we can connect an analog input which varies (e.g. between 0V and 3.3V) and obtain a digital representation of this value (e.g. between 0 and 4095) [14].

1.5.1 Questions

1. What is an `Analog Input`? Give an example of real application to explain the different usage of `Digital Input` and `Analog Input`.
2. What is the purpose of an ADC? What are the steps of an ADC?
3. What are the characteristics of ESP32's ADC? (Explain how you got the answer.)
4. What is the purpose of a voltage divider? (See [22] for help...)

1.5.2 Practice

Here, we will use a photoresistor and translate its resistance into a digital signal.

> A photoresistor or **L**ight-**D**ependent **R**esistor (LDR) or photocell is a light-controlled variable resistor. The resistance of a photoresistor decreases with increasing incident light intensity; in other words, it exhibits photoconductivity.
>
> *Wikipedia – Photoresistor*

Create a program that:

1. Convert the value obtained using a LDR to a value in the range 0-4095, and display it every 1 second on your computer using the serial connection.
   You have to use a simple voltage divider circuit to convert the variable resistor of the LDR to a variable voltage. Then, connect the output of your voltage divider circuit to an analog input of your board (e.g. `A0` in code, or `ADC1 CH0` on the board, or `GPIO 36`.). Give details in your report.
2. Add a calibration step at the beginning of your program in order to obtain values between 0 and 255. You can use this page: `https://docs.arduino.cc/built-in-examples/analog/Calibration/`. During this calibration step, you will light up the LDR (for example with the flash of your phone) and you will hide the LDR (for example with your hand) to obtain min/max values. Do not forget to give explanations in your report.
3. Repeat the previous question using a potentiometer instead of the LDR.

1.5.3 To go further

Here is the "real" engineering part... read the datasheet/document and **compute/display the actual values** according to your own configuration.

For example, imagine you have a Current Clamp which allows measuring the current (See [18, 7] and Figure 1.1). The Current Clamp will not give you the exact consumption, depending on the characteristics (maximum amperage, etc.), you will have to compute the actual consumption of current/energy. An engineer must know what equipment to use but, above all, he must know how to use it according to its characteristics and its technical datasheet.

So, let's start slowly... with a simple temperature sensor and your LDR.

1. First, we will use a simple temperature sensor, the `TMP36` or `LM35` model. Find the datasheet of this sensor, connect it to your board and display the value of the temperature on your computer. On your report, you will explain the formula used to compute the temperature according to the result of the ADC and give the page in the datasheet where you find the information.
2. Great, you know how to read a datasheet! Now... go back to the LDR, find the Datasheet and:



Figure 1.1: Example of Current Clamp used in industry to measure current/energy consumption.

- According to your voltage divider theory, the value of the fixed resistor used and the voltage read with your ADC: display the actual resistance value of the LDR.
- According to the datasheet of the LDR and the value of the resistance computed previously: compute and display the value of the receiving luminosity (the unit of measure for light is `lux`).

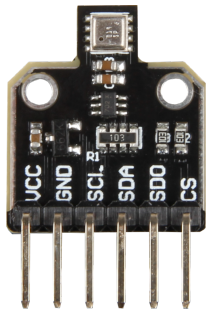
PART 2 | ACQUIRE EXTERNAL DATA, PUSH DATA TO THE WORLD

By the end of this part, you must be able to explain, in your own words:

- what distinguishes I2C from SPI, and how you choose between them;
- what the configuration lines of the BME680 actually do;
- how you locate the right field in a JSON response returned by an API;
- what the publish/subscribe principle changes compared with a direct request.

These are the concepts assessed in Quiz 2, at the start of Lab Session 3.

2.1 Using I2C or SPI to acquire or push data



(a) SEN-BME680 (from Joy-It).



(b) LCD (2 lines, 16 characters) with I2C adapter (from Joy-It).

Figure 2.1: Some components using I2C or SPI to communicate.

Inter-Integrated Circuit (I2C) is "a serial communication bus widely used for attaching lower-speed peripheral to microcontroller" [19] and Serial Peripheral Interface (SPI) is "a synchronous serial communication interface specification used for short-distance communication, primarily in embedded systems" [20]. Many components can communicate in I2C or SPI and the fact that it is a bus allows using few wires to communicate with many peripherals.

2.1.1 Play with the SEN-BME680 to acquire external data

We will use a SEN-BME680 (from Joy-It) with the ESP32. This component can communicate with I2C or SPI.

First, you have to install the library `Adafruit BME680 Library` by *Adafruit*. You will then have examples in your Arduino IDE.

Using Annexe 5.2, create a program on your ESP32 to display on the Serial output:

1. the current temperature (in °Celius and in °Fahrenheit) and the current humidity

2.1.2 Play with the LCD screen to push data

We will use a I2C LCD screen to display information with the ESP32. This component can communicate with I2C using a adapter directly connected to the LCD screen.

First, you have to install the library `LiquidCrystal I2C` by *Marco Schwartz* or *Frank de Brabander* available here. You will then have examples in your Arduino IDE.

Create a program on your ESP32 to display:

1. a simple message with two lines:
   - First line: "Great IoT Tutorial!"

- Second line: "Launched for Xs", with "X" increases each second.

2. the current temperature and the current humidity taken from the BME680

2.2 Using API to acquire external data

In this part, you will connect your ESP32 to external sources of data through API [17]. A lot of services offer API to access to their data, you can explore `https://rapidapi.com/` to find some example. Here are some specific examples:

- For RTE (Réseau de Transport d'Électricité), a lot of useful API are proposed: `https://data.rte-france.com/`.
- For each local energy consumption meter, you have an API to be able to gather its data. e.g. a CGE EcoRT2, according to its documentation, you will be able to create your own program by querying it to fetch its data from its API.
- If you need the current or forecast weather, check services such as OpenWeatherMap, AccuWeather or Weatherstack.
- If you have connected light in your home such as Philips Hue, you can use their API to directly control them.
- If you have an online music account such as Spotify, you can use their API to play music in specific context.

2.2.1 Get Weather data on ESP32

We will use a free account on Weatherstack to learn how to call and extract data from API. First, you might consider examples to understand how to connect your ESP32 to WiFi hotspots (see ESP32 WiFiClient example on your Arduino IDE) and how to call a web page on Arduino (See ESP32 BasicHttpClient example on your Arduino IDE) and how to read a JSON (See ArduinoJson library for example.) Or you can simply read the proposed code in Annexe 5.3 if you read this line before starting to code...

1. Create free account on `https://weatherstack.com` and choose a location (Paris? New York?)
2. Connect your ESP32 board to Internet, e.g., to your smartphone wifi hotspot.
3. According to the documentation, create a program on your ESP32 to display:
   - name of your city, current time, current temperature and wind speed

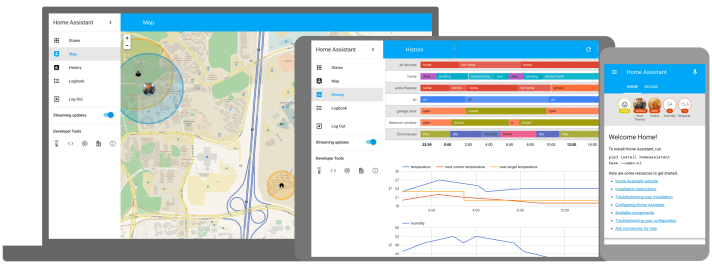
2.3 Push data to the world

In this part, you will connect your boards to an external service to send data. It is possible to send data to a database, to other local devices or to API. We will use the free environment Adafruit IO to experiment this idea. Note that with the same service, you can also receive data (by subscribing to "feeds"). You have a full code example on Annexe 5.4.

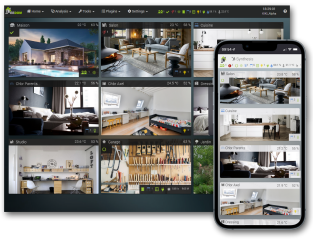
1. Create a free account on `https://io.adafruit.com/`
2. Connect your ESP32 board to Internet, e.g., to your smartphone wifi hotspot.
3. Based on tutorials [1], push data (e.g. data from a temperature sensor?) to the AdafruitIO Cloud.

2.4 To go further (not evaluated)

You love to play with data and you want to imagine a way to automate some things in your home?
Take a look at OpenSource projects such as Jeedom or Home Assistant. It is quite simple to install, e.g., on a RaspberryPi...



(a) The frontend of Home Assistant. Image source.



(b) The frontend of Jeedom. Image source.

Figure 2.2: Some great OpenSource projects for home automation.

PART 3 | ESP32 AND ENERGY CONSUMPTION

By the end of this part, you must be able to explain, in your own words:

- why measuring the voltage across a shunt resistor gives access to the current;
- how the shunt degrades the voltage actually available to the ESP32, and from what value it becomes a problem;
- over what time window an average current must be integrated, and why the choice of window changes the result;
- what light sleep and deep sleep switch off respectively, and what each of them costs on waking;
- what has to be done for a variable to survive a deep sleep;
- **why your measured current in deep sleep is two to three orders of magnitude above the datasheet figure.**

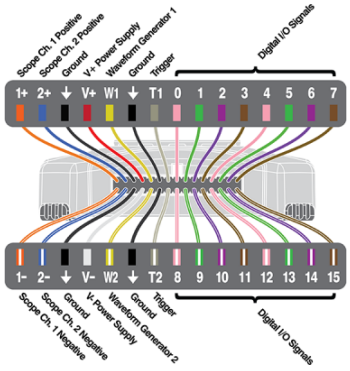
This part is the core of Quiz 3, at the start of Lab Session 4.

The Analog Discovery 2 is only available during this session. Whatever you have not captured by the end of Lab Session 3 cannot be captured later.
Fill in the **measurement sheet** available on Moodle as you go, and deposit it, exported to PDF, **before leaving the room** — even if it is incomplete. An empty box with its explanation is worth more than a value reconstructed afterwards. You will then attach that same sheet, unchanged, as section A of your report.

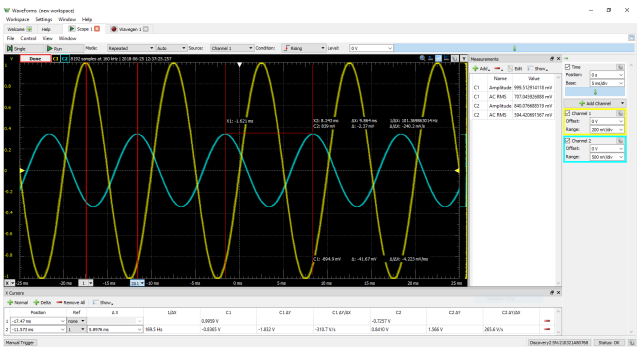
3.1 Measuring energy consumption with Analog Discovery 2



(a) Analog Discovery 2 (from Digilent).



(b) Analog Discovery 2 PINOUT Reference [3]



(c) Oscilloscope frontend on Diligent WaveForms software.

Figure 3.1: Diligent hardware and software.

We will use Analog Discovery 2 and WaveForms mainly to:

1. Supply the electronic assembly with a controlled voltage.
2. Measure the voltage across known resistors to … calculate a current.
3. Calculate an average current over a period of time in order to calculate an average consumption.

3.1.1 Installation of Diligent WaveForms

First of all, we need to install the software "Diligent WaveForms" which will allow us to control and view data from the Analog Discovery.

Follow the instruction to download and install the Diligent WaveForms software on your computer: https://digilent.com/shop/waveforms/ (Windows, Linux, MacOS).

3.1.2 Installation of your hardware setup

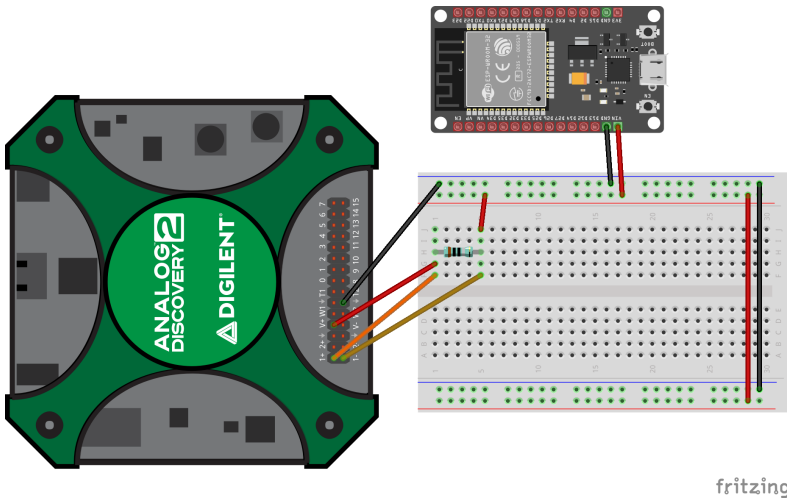


Figure 3.2: Example of hardware installation to power and measure the consumption of an ESP32 with the Analog Discovery 2.

Power supply the assembly

Now, you will connect your Analog Discovery 2 to supply your assembly.

1. See Figure 3.2 to wire the power supply.
   Some precisions:
   - We use V^+ and ↓ (GND pin of the Analog Discovery 2) pins which will deliver the power supply.
   - We **shall not forget** to connect the pins ↓ with GND of the ESP32 so each element has the same voltage reference.
2. See the Diligent WaveForms documentation [6] to configure a power supply with $V^+ = 5V$.

3.1.3 Measuring a current... with a resistor

We will use a resistor with a small resistance ($\simeq 1\Omega$) to measure the current drawn by our ESP32. The following is based on a tutorial from Diligent documentation [4].

1. See Figure 3.2 to wire the oscilloscope channel.
   Some precisions:
   - We use 1^+ and 1^- pins which correspond to the first oscilloscope channel.
   - Voltage is measured across the resistance.
2. See the Diligent WaveForms documentation [5] to configure the oscilloscope channel display.
3. Explain why it is interesting to measure the voltage at this point.
   You can respond to these questions if it helps you:
   (a) According to the datasheet of our ESP32 board, what is the maximum current drawn by the board?
   (b) According to the previous answer and the value of the resistance used, what will be the maximum voltage across the resistance?
   (c) According to the previous answer and the datasheet of our ESP32 board, what will be the minimum voltage available to our ESP32 board? Is it enough to keep it running?
   (d) Can we calculate the current flowing through the resistor? Is this current different from that received by

our ESP32 board?
   (e) If we have the current drawn by our ESP32 board as well as its supply voltage, is it possible to know how much it consumes?
4. Using the Diligent WaveForms documentation [4], add a $Math\ channel$ to compute and display the current drawn by our ESP32 board.
5. **Oh... wait...** are you sure the resistance is exactly 1Ω?! Take a moment, use a multimeter to check the real value of your resistance... and update your formulae.
   Write that measured value on your measurement sheet. Every current you compute from now on depends on it, and it is unique to your pair: no other pair has exactly the same resistor.

3.2 Using Sleep and DeepSleep modes

3.2.1 Normal Mode - Consumption

To see a difference... we need a reference.

1. Upload a code to blink a LED with **your own pair parameters**: T_{on} on, then off for the rest of a 2000 ms period (see page 2).
2. Measure the average current drawn. Report it on your measurement sheet.
   Explain on which period of time you decide to measure this average and why.
3. With a battery of 1000mAh, how long will it last?

3.2.2 Light Sleep Mode - Consumption

1. Change your code to blink a LED with the same parameters, using a Light Sleep mode.
2. Measure the average current used. Report it on your measurement sheet and compare it with the "Normal" mode.
3. With a battery of 1000mAh, how long will it last?

3.2.3 DeepSleep Mode - Consumption

1. Change your code to blink a LED using a DeepSleep mode, waking up every $T_{sleep} = 1 + 0.25 \times N$ seconds (see page 2).
2. Measure the average current used. Report it on your measurement sheet, compare it with the "Normal" and "Sleep" mode, and explain what you observed.
3. With a battery of 1000mAh, how long will it last?
4. **Compare your measured value with the datasheet.** The manufacturer announces a current of the order of 10 μA in deep sleep. What order of magnitude did *you* measure? Write the gap down on your measurement sheet, even if you cannot explain it yet.

3.2.4 A few tests and measurements

Response time with a button - Light Sleep Mode

1. Add a button (or use the Digital I/O of the Analog Discovery 2) and upload a code which will light up a LED while the button is pressed.
2. Change your wiring to observe on the oscilloscope:
   - when the (input) button is pressed.
   - when the (output) LED is lighted up.
3. Measure that response time and report it on your measurement sheet.
4. Find a way to use a Light Sleep mode, reduce the response time and optimize the consumption.

Counting the boots - DeepSleep Mode

1. Add a **boot counter** to your serial prefix, so that each line reads `[Binome NN - NAME1 & NAME2] boot #3 - ...`. The counter must be incremented at every start-up. Implement it with an ordinary global variable first.
2. Run your deep-sleep program for several cycles and report what the counter shows. Explain what you observe.
3. `esp_reset_reason()` tells you why the board started. Print it next to your counter. What does it return after a power-up, and what does it return after a deep-sleep wake-up? What does that tell you about what a deep sleep actually is?
4. Make the counter survive a deep sleep. Explain where the value is now stored, why that memory survives while your global variable does not, and what it costs.
5. Would the same solution work after a hardware reset, or after the power supply is disconnected? Justify.

Response time with an ADC - DeepSleep Mode

1. Add a LDR and upload a code which will light up a LED while the LDR is higher than a specific value.
2. Change your wiring to observe on the oscilloscope:
   - when the (input) LDR is higher than a specific value
   - when the (output) LED is lighted up
3. Measure that response time and report it on your measurement sheet.
4. Find a way to use a DeepSleep mode, reduce the response time and optimize the consumption

3.2.5 To go further ...

If you want to go further and better understand certain parts, do not hesitate to consult the references: [23, 2, 13, 16, 24, 25]

Part 4 | Need for specifications to optimize consumption

By the end of this part, you must be able to explain, in your own words:
- what questions must be asked before choosing an architecture;
- how an acceptable response time constrains the sleep strategy.

This part is not assessed by a quiz: it feeds directly into the specification document of your PoC.

4.1 What will be used? When? What response time is acceptable?

4.1.1 Context

Imagine a project: I want to measure the humidity of a green wall and, depending on its humidity, the expected rain and the temperature expected for the day, I want to activate a sprinkler pump so that the humidity does not not fall below a threshold during the day.

4.1.2 Questions to ask... to optimize the energy consumption

Your answers to this section are the starting point of the **specification and measurement plan** of your PoC, to be deposited before Lab Session 5: which quantities you will measure, with what protocol, and what battery life you are aiming for. Your final PoC report will then confront what you planned with what you measured.

1. What questions should I ask?
   Suggest several categories of questions and a few questions for each.
2. What global architectures are possible to optimize the overall consumption of my system?
   Suggest at least two global architectures and specify the advantages and disadvantages of each.

PART 5 | ANNEXES

5.1 Details on ADC and ESP32

The ESP32's ADC is very configurable. It is essential to consult this page: https://docs.espressif.com/projects/esp-idf/en/v4.2.3/esp32/api-reference/peripherals/adc.html

5.1.1 Constraints on ADC2 and Wifi

You can read there that:

> *However, the usage of ADC2 has some restrictions for the application:*
>
> 1. *ADC2 is used by the Wi-Fi driver. Therefore the application can only use ADC2 when the Wi-Fi driver has not started.*
> 2. *Some of the ADC2 pins are used as strapping pins (GPIO 0, 2, 15) thus cannot be used freely.*

So, for example, **if you use a Wifi connection** to connect to Adafruit IO, you will not be able to use the ADC2 (so **you won't be able to use GPIOs** 0, 2, 4, 12 - 15 and 25 - 27)

5.1.2 Configuration of ADC and full-scale voltage

You can read there that:

Attenuation	Parameter	Measurable Voltage
$0dB$	ADC_ATTEN_DB_0	$100mV$ $950mV$
$2.5dB$	ADC_ATTEN_DB_2_5	$100mV$ $1250mV$
$6dB$	ADC_ATTEN_DB_6	$150mV$ $1750mV$
$11dB$	ADC_ATTEN_DB_11 or ADC_ATTEN_DB_12	$150mV$ $3100mV$

Figure 5.1: Configuration of ESP32 ADC according to ADC Arduino ESP32.

So if the voltage you want to read is less than $950mV$, you can set the attenuation to $0dB$, so you will have maximum accuracy (4096 values) between $100mV$ and $950mV$. E.g., if you have a TMP36 temperature sensor that gives an output voltage of ($500mV + 10mV \times TemperatureInCelsius$), then you will have $500mV$ for $0°C$ and $950mV$ for $45°C$.

If you want to measure a temperature in this range, you can therefore set an attenuation of your ADC to $0db$ and you will have a reading precision of approximately $((950 - 100)/4095)$ $0.21mV$, or $(0.21/10)$ $0.021°C$ (to be compared with the inherent precision of this TMP36 sensor).

See the code `ESP32_Arduino_ADC` given on my GitHub.

5.2 ESP32 with BME680

5.2.1 Using I2C

You may use the example `bme680test` with the I2C connection and the definition of the GPIO used, see the code `ESP32_Arduino_BME` given on my GitHub.



If you have problems when you program your board, try to disconnect the $3V3/VCC$ pin during the programming part, then connect it again and restart your ESP32 board (using button EN).

BME680 pin	ESP32 GPIO
VCC	$3V3$
GND	GND
SCL/SCK	22
SDA/SDI	21

Figure 5.2: I2C connection between BME680 and ESP32.



By default, the I2C address of the BME680 is $0x77$. If the address has changed, you have to:
- change the line `if (!bme.begin())`
- with `if (!bme.begin(NEW_ADDRESS))`

See here for details.

5.2.2 Using SPI

You may use the example `bme680test` with the SPI connection and the definition of the GPIO used, see the code `ESP32_Arduino_BME` given on my GitHub.

BME680 pin	ESP32 GPIO
VCC	$3V3$
GND	GND
SCL/SCK	14
SDA/SDI	13
SDO	12
CS	15

Figure 5.3: SPI connection between BME680 and ESP32.

5.3 ESP32 with API call and JSON read

First, you have to install the library `ArduinoJson` by $Benoit Blanchon$. You will then have examples in your Arduino IDE. Then, you can consider the code `ESP32_Arduino_HTTP_WeatherStack` given on my GitHub and test using your own `MY_API_KEY` from WeatherStack.

5.4 ESP32 and AdafruitIO

First, you have to install the library `Adafruit IO Arduino` by $Adafruit$. You will then have examples in your Arduino IDE. Then, you can consider the code `ESP32_Arduino_MQTT_PubSub` given on my GitHub and test using your own `MQTT_USERNAME` and `MQTT_KEY` from your AdafruitIO account.

5.5 ESP32 and ULP

If you want to create a code with ULP support (e.g., ADC in Deep Sleep mode), you have to code in assembler... you can:
- **The simplest way, but more limited... and not working with last version of ESP-Arduino**. Use the C macro proposed by Espressif [11].
- **If you want to directly code in assembler:**
  - On Arduino IDE... it is possible, but complex.
  - On PlaformIO, you can directly code using the espidf framework. See example `ESP32_ESPDIF_Timer_ADC_ulp` given on my GitHub.

Some useful links:
- ESP32 ULP Coprocessor Instruction Set [10]
- ESP32 GPIO& RTC GPIO pin numbering mapping [9].

Part 6 | Bibliography

[1] AdafruitIO. *Arduino and Adafruit IO (Broken?)* URL: https://learn.adafruit.com/welcome-to-adafruit-io/arduino-and-adafruit-io.

[2] Projets DIY. *ESP32. Code Arduino pour mise en sommeil (Deep Sleep) et réveils (Timer, Touch Pad, GPIO...) (Broken?)* URL: https://projetsdiy.fr/esp32-code-arduino-sommeil-deep-sleep-reveils-timer-touch-pad-gpio/.

[3] Diligent. *Analog Discovery 2 Specifications*. URL: https://digilent.com/reference/test-and-measurement/analog-discovery-2/specifications.

[4] Diligent. *Measuring Current*. URL: https://digilent.com/reference/test-and-measurement/analog-discovery-2/demos/measuring-current?redirect=1.

[5] Diligent. *Using the Oscilloscope*. URL: https://digilent.com/reference/test-and-measurement/guides/waveforms-oscilloscope.

[6] Diligent. *Using the Power Supplies*. URL: https://digilent.com/reference/test-and-measurement/guides/waveforms-supplies.

[7] GCE Electronics. *Pinces pour module X400-CT + EcoRT2*. URL: http://gce-electronics.com/fr/capteurs/806-pinces-amperemetriques-pour-module-x400-ct.html.

[8] Last Minute Engineers. *ESP32 Pinout Reference*. URL: https://lastminuteengineers.com/esp32-pinout-reference/.

[9] Espressif. *ESP32 GPIO & RTC GPIO*. URL: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/gpio.html.

[10] Espressif. *ESP32 ULP Coprocessor Instruction Set*. URL: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/ulp_instruction_set.html.

[11] Espressif. *Programming ULP coprocessor using C macros (legacy)*. URL: https://docs.espressif.com/projects/esp-idf/en/v4.2.3/esp32/api-guides/ulp_macros.html.

[12] Serial/UART Function. *UART – Universal asynchronous receiver-transmitter*. URL: https://www.arduino.cc/reference/en/language/functions/communication/serial/.

[13] LastMinuteEngineers. *Insight Into ESP32 Sleep Modes & Their Power Consumption*. URL: https://lastminuteengineers.com/esp32-sleep-modes-power-consumption/.

[14] RandomNerdTutorials. *ESP32 ADC – Read Analog Values with Arduino IDE*. URL: https://randomnerdtutorials.com/esp32-adc-analog-read-arduino-ide/.

[15] RandomNerdTutorials. *Installing the ESP32 Board in Arduino IDE (Windows, Mac OS X, Linux)*. URL: https://randomnerdtutorials.com/installing-the-esp32-board-in-arduino-ide-windows-instructions/.

[16] Circuit Rocks. *FreeRTOS: ESP32 Deep Sleep*. URL: https://learn.circuit.rocks/esp32-deep-sleep.

[17] Wikipedia. *API*. URL: https://en.wikipedia.org/wiki/API.

[18] Wikipedia. *Current Clamp*. URL: https://en.wikipedia.org/wiki/Current_clamp.

[19] Wikipedia. *I2C (Inter-Integrated Circuit)*. URL: https://en.wikipedia.org/wiki/I%C2%B2C.

[20] Wikipedia. *SPI (Serial Peripheral Interface)*. URL: https://en.wikipedia.org/wiki/Serial_Peripheral_Interface.

[21] Wikipedia. *UART – Universal asynchronous receiver-transmitter*. URL: https://en.wikipedia.org/wiki/Universal_asynchronous_receiver-transmitter.

[22] Wikipedia. *Voltage divider*. URL: https://en.wikipedia.org/wiki/Voltage_divider.

[23] diyiot. *Guide to Reduce the ESP32 Power Consumption by 95%*. URL: https://diyi0t.com/reduce-the-esp32-power-consumption/.

[24] educ8s. *ESP32 Deep Sleep Tutorial*. URL: https://www.instructables.com/ESP32-Deep-Sleep-Tutorial/.

[25] educ8s. *ESP32 E-Paper Thermometer*. URL: https://www.instructables.com/ESP32-E-Paper-Thermometer/.